On the Number of Distinct Topological Bases of a Finite Set of Size N

Lars Warren Ericson[1]

[1]Catskills Research Company
[1]https://github.com/catskillsresearch/scott_models/tree/main/CARDB
[1]lars.ericson@catskillsresearch.com

August 18, 2026

ORCID: 0000-0001-8299-9361

Abstract

For a finite set S with $|S| = N$, the number of families $\mathcal{B} \subseteq \mathcal{P}(S)$ that are topological bases is $\#(N) = \sum_{\mathcal{T} \in \mathrm{Top}(S)} 2^{|\mathcal{T}|-|\mathcal{M}_\mathcal{T}|}$, where $\mathcal{M}_\mathcal{T}$ is the canonical minimal basis of minimal open neighborhoods. The identity is proved in Lean 4 / Mathlib (`CARDB.lean`): bases generating $\mathcal{T}$ are exactly the sets with $\mathcal{M}_\mathcal{T} \subseteq \mathcal{B} \subseteq \mathcal{T}$. Companion artifact: `CARDB.lean` (standalone Lake package in this directory).

Contents

1 Introduction and Definitions

Let S be a set. In point-set topology, a family $\mathcal{B} \subseteq \mathcal{P}(S)$ is defined as a **basis for a topology on** S if it satisfies two axioms:

1. **Covering Axiom:**

$$\bigcup_{B\in\mathcal{B}} B = S \quad \iff \quad \forall x \in S,\ \exists B \in \mathcal{B} \text{ such that } x \in B$$

2. **Intersection Axiom:**

$$\forall B_1, B_2 \in \mathcal{B},\ \forall x \in B_1 \cap B_2,\ \exists B_3 \in \mathcal{B} \text{ such that } x \in B_3 \subseteq B_1 \cap B_2$$

The topology generated by $\mathcal{B}$, written $\mathrm{gen}(\mathcal{B})$ (Mathlib: `generateFrom`), is the collection of all arbitrary unions of subfamilies of $\mathcal{B}$:

$$\mathrm{gen}(\mathcal{B}) = \left\{ \bigcup \mathcal{C} \;\middle|\; \mathcal{C} \subseteq \mathcal{B} \right\}$$

We reserve $\mathcal{T}$ for a topology on S, never for this generation map.

Companion file: `CARDB.lean`.

```
import Mathlib.Topology.AlexandrovDiscrete
import Mathlib.Data.Fintype.BigOperators
import Mathlib.Data.Fintype.Powerset
import Mathlib.Data.Set.Card
import Mathlib.Data.Set.Finite.Basic
import Mathlib.Logic.Equiv.Sum

open Set TopologicalSpace
open scoped BigOperators

variable {α : Type*} [Fintype α] [DecidableEq α]

/-- A collection of subsets `B` is a topological basis if it covers `α`
    and satisfies the local intersection property. -/
def IsBasis (B : Set (Set α)) : Prop :=
  (⋃₀ B = univ) ∧
  ∀ {{U V}}, U ∈ B → V ∈ B → ∀ x ∈ U ∩ V, ∃ W ∈ B, x ∈ W ∧ W ⊆ U ∩ V
```

1.1 Set-Theoretic Status

The definition of a basis and the topology generated by it is strictly constructible within standard Zermelo–Fraenkel set theory (ZF) without the Axiom of Choice: - $\mathcal{P}(S)$ exists by the **Power Set Axiom**. - $\mathcal{B}$ exists as a subset of $\mathcal{P}(S)$ by **Separation**. - For each $\mathcal{C} \in \mathcal{P}(\mathcal{B})$, $\bigcup \mathcal{C}$ exists by the **Union Axiom**. - The collection $\mathrm{gen}(\mathcal{B})$ is the image of $\mathcal{C} \mapsto \bigcup \mathcal{C}$ under the **Axiom of Replacement**.

---

2 The Enumeration Problem

Let S be a finite set with $|S| = N$. We wish to determine the exact quantity:

$$\#(S) = \left| \{ \mathcal{B} \subseteq \mathcal{P}(S) \mid \mathcal{B} \text{ is a topological basis on } S \} \right|$$

The objects being counted are the subtype of families satisfying `IsBasis`. Because α is finite, there are only finitely many such families, and only finitely many topologies: a topology is recovered from its set of opens, and that assignment is injective.

```
def ValidBasis (α : Type*) [Fintype α] := { B : Set (Set α) // IsBasis B }

/-- Open sets of `t`, as a family of subsets. This map is injective, so there
    are only finitely many topologies on a finite type. -/
def opensOf (t : TopologicalSpace α) : Set (Set α) := {U | t.IsOpen U}

omit [Fintype α] [DecidableEq α] in
theorem opensOf_injective :
    Function.Injective (opensOf : TopologicalSpace α → Set (Set α)) :=
  leftInverse_generateFrom.injective

noncomputable instance : DecidableEq (TopologicalSpace α) :=
  opensOf_injective.decidableEq

noncomputable instance : Fintype (TopologicalSpace α) :=
  Fintype.ofInjective opensOf opensOf_injective

noncomputable instance : DecidablePred (IsBasis : Set (Set α) → Prop) :=
  fun _ => Classical.dec _

noncomputable instance : Fintype (ValidBasis α) :=
  Subtype.fintype _
```

At first inspection, this appears to be a local hypergraph enumeration problem. However, the basis axioms enforce global closure properties that reduce the count to the classification of finite Alexandrov spaces.

---

3 Structural Decomposition and the Exact Identity

3.1 Finite Topologies are Alexandrov

For any finite set S, any topology $\mathcal{T}$ on S is closed under arbitrary intersections (since every intersection of open sets is finite). Consequently, for every point $x \in S$, there exists a unique **minimal open neighborhood**:

$$U_x = \bigcap \{U \in \mathcal{T} \mid x \in U\}$$

The collection:

$$\mathcal{M}_{\mathcal{T}} = \{U_x \mid x \in S\}$$

is called the **canonical minimal basis** of $\mathcal{T}$. In Mathlib this is the neighborhoods kernel `nhdsKer {x}` (`Mathlib.Topology.NhdsKer`); finiteness supplies `AlexandrovDiscrete`, so `isOpen_nhdsKer` applies.

```
variable (t : TopologicalSpace α)

/-- In a finite topological space, the minimal open neighborhood of `x`
    (Mathlib's `nhdsKer {x}` at topology `t`). -/
abbrev minimalOpen (x : α) : Set α :=
  @nhdsKer α t {x}

/-- The minimal basis `M_T` is the collection of all minimal open neighborhoods. -/
def minimalBasis : Set (Set α) :=
  range (minimalOpen t)
```

By construction $x \in U_x$, and if U is open with $x \in U$ then $U_x \subseteq U$. On a finite space U_x is open, and the U_x generate $\mathcal{T}$: every open U is $\bigcup_{x \in U} U_x$.

```
omit [Fintype α] [DecidableEq α] in
theorem mem_minimalOpen_self (x : α) : x ∈ minimalOpen t x :=
  @subset_nhdsKer α t {x} x (mem_singleton x)

omit [Fintype α] [DecidableEq α] in
theorem minimalOpen_subset_of_isOpen {x : α} {U : Set α}
    (hU : t.IsOpen U) (hx : x ∈ U) : minimalOpen t x ⊆ U :=
  @nhdsKer_minimal α t {x} U (singleton_subset_iff.mpr hx) hU

omit [DecidableEq α] in
/-- Finite spaces are Alexandrov-discrete, so `nhdsKer {x}` is open. -/
theorem isOpen_minimalOpen (x : α) : t.IsOpen (minimalOpen t x) :=
  letI := t
  isOpen_nhdsKer

omit [DecidableEq α] in
/-- The minimal basis alone generates the topology `t`. -/
theorem generateFrom_minimalBasis : generateFrom (minimalBasis t) = t := by
  ext U
  constructor
  · intro h
    letI := t
    induction h with
    | basic s hs =>
      obtain ⟨x, rfl⟩ := hs
      exact isOpen_minimalOpen t x
    | univ => exact isOpen_univ
    | inter _ _ _ _ hs ht => exact hs.inter ht
    | sUnion S _ hS => exact isOpen_sUnion fun s hs => hS s hs
  · intro hU
    have : U = ⋃₀ (minimalOpen t '' U) := by
      ext y
      constructor
      · intro hy
        exact ⟨minimalOpen t y, ⟨y, hy, rfl⟩, mem_minimalOpen_self t y⟩
      · rintro ⟨_, ⟨x, hx, rfl⟩, hy⟩
        exact minimalOpen_subset_of_isOpen t hU hx hy
    rw [this]
    exact GenerateOpen.sUnion _ fun V hV =>
      let ⟨x, _, hx⟩ := hV
      hx ▸ GenerateOpen.basic _ (mem_range_self x)
```

3.2 Fiber Decomposition Theorem

Let $\mathrm{Top}(S)$ be the set of all topologies on S. The assignment $\mathcal{B} \mapsto \mathrm{gen}(\mathcal{B})$ defines a surjective, many-to-one function from the set of all valid bases to $\mathrm{Top}(S)$. For a topology $\mathcal{T}$, the **fiber** over $\mathcal{T}$ is the preimage of that point,

$$\{\mathcal{B} \mid \mathrm{gen}(\mathcal{B}) = \mathcal{T}\},$$

i.e. the valid bases that generate exactly $\mathcal{T}$. The count $\#(S)$ is the sum of the cardinalities of these (pairwise disjoint) fibers.

```
/-- Valid bases that generate exactly `t`: the fiber of `generateFrom` over `t`. -/
abbrev Fiber (t : TopologicalSpace α) :=
  { B : ValidBasis α // generateFrom B.1 = t }
```

If $\mathcal{B}$ satisfies the two basis axioms, then Mathlib's `IsTopologicalBasis` holds for the topology `generateFrom B`:

```
omit [Fintype α] [DecidableEq α] in
/-- `IsBasis` plus `generateFrom B = t` is Mathlib's `IsTopologicalBasis`. -/
theorem isTopologicalBasis_generateFrom {B : Set (Set α)} (hB : IsBasis B) :
    @IsTopologicalBasis α (generateFrom B) B := by
```

```
  letI := generateFrom B
  refine ⟨?_, hB.1, rfl⟩
  intro t₁ ht₁ t₂ ht₂ x hx
  exact hB.2 ht₁ ht₂ x hx
```

Lemma (Characterization of Basis Fibers): Let $\mathcal{T}$ be a topology on a finite set S. A collection $\mathcal{B} \subseteq \mathcal{P}(S)$ satisfies $\mathrm{gen}(\mathcal{B}) = \mathcal{T}$ (and is therefore a valid basis) if and only if:

$$\mathcal{M}_{\mathcal{T}} \subseteq \mathcal{B} \subseteq \mathcal{T}$$

Proof.

- **Necessity:** If $\mathrm{gen}(\mathcal{B}) = \mathcal{T}$, then every element of $\mathcal{B}$ is open in $\mathcal{T}$, so $\mathcal{B} \subseteq \mathcal{T}$. Furthermore, for every $x \in S$, the minimal neighborhood $U_x \in \mathcal{T}$ must be a union of elements of $\mathcal{B}$. Thus, there exists $B_\alpha \in \mathcal{B}$ containing x with $B_\alpha \subseteq U_x$. By minimality of U_x, we have $U_x \subseteq B_\alpha$, forcing $B_\alpha = U_x$. Hence, $U_x \in \mathcal{B}$, so $\mathcal{M}_{\mathcal{T}} \subseteq \mathcal{B}$.
- **Sufficiency:** If $\mathcal{M}_{\mathcal{T}} \subseteq \mathcal{B} \subseteq \mathcal{T}$, then since $\mathcal{T}$ is closed under unions, $\mathrm{gen}(\mathcal{B}) \subseteq \mathcal{T}$. Conversely, any open set $U \in \mathcal{T}$ can be written as $U = \bigcup_{x \in U} U_x$, which is a union of elements from $\mathcal{M}_{\mathcal{T}} \subseteq \mathcal{B}$. Thus $U \in \mathrm{gen}(\mathcal{B})$, so $\mathcal{T} \subseteq \mathrm{gen}(\mathcal{B})$. □

```
omit [DecidableEq α] in
/-- Core lemma: `B` generates `t` iff `B` contains the minimal basis
    and only contains open sets of `t`. -/
theorem isBasis_and_generates_iff (B : Set (Set α)) :
    (IsBasis B ∧ generateFrom B = t) ↔
    (minimalBasis t ⊆ B ∧ B ⊆ {U | t.IsOpen U}) := by
  constructor
  · rintro ⟨hB, rfl⟩
    constructor
    · rintro _ ⟨x, rfl⟩
      letI := generateFrom B
      have hb : IsTopologicalBasis B := isTopologicalBasis_generateFrom hB
      have hx : x ∈ minimalOpen (generateFrom B) x := mem_minimalOpen_self _ x
      have hUo : IsOpen (minimalOpen (generateFrom B) x) := isOpen_minimalOpen _ x
      obtain ⟨W, hW, hxW, hWU⟩ := hb.exists_subset_of_mem_open hx hUo
      have hUW : minimalOpen (generateFrom B) x ⊆ W :=
        minimalOpen_subset_of_isOpen _ (isOpen_generateFrom_of_mem hW) hxW
      rwa [← Subset.antisymm hWU hUW]
    · intro U hU
      exact isOpen_generateFrom_of_mem hU
  · rintro ⟨h_min, h_sub⟩
    constructor
    · constructor
      · ext x
        simp only [mem_sUnion, mem_univ, iff_true]
        exact ⟨minimalOpen t x, h_min ⟨x, rfl⟩, mem_minimalOpen_self t x⟩
      · intro U V hU hV x hx
        refine ⟨minimalOpen t x, h_min ⟨x, rfl⟩, mem_minimalOpen_self t x, ?_⟩
        intro y hy
        exact ⟨minimalOpen_subset_of_isOpen t (h_sub hU) hx.1 hy,
          minimalOpen_subset_of_isOpen t (h_sub hV) hx.2 hy⟩
    · refine le_antisymm ?_ ?_
      · exact generateFrom_anti h_min |>.trans (generateFrom_minimalBasis t).le
      · exact le_generateFrom_iff_subset_isOpen.2 h_sub
```

3.3 The Exact Formula

Because the minimal basis elements $\mathcal{M}_{\mathcal{T}}$ are mandatory, any valid basis generating $\mathcal{T}$ is formed by taking $\mathcal{M}_{\mathcal{T}}$ and freely adjoining any subset of the "redundant" open sets in $\mathcal{T} \setminus \mathcal{M}_{\mathcal{T}}$ (including $\emptyset$, which never affects the basis conditions since $\emptyset \notin \mathcal{M}_{\mathcal{T}}$).

Therefore, the fiber over $\mathcal{T}$ has size:

$$|\{\mathcal{B} \mid \text{gen}(\mathcal{B}) = \mathcal{T}\}| = 2^{|\mathcal{T}|-|\mathcal{M}_\mathcal{T}|}$$

The type `Fiber t` is equivalent to the power set of the redundant opens $\mathcal{T} \setminus \mathcal{M}_\mathcal{T}$:

```
/-- The fiber over a topology `t` is equivalent to the power set
    of the redundant open sets. -/
def fiberEquiv (t : TopologicalSpace α) :
    Fiber t ≃
      Set { U : Set α // t.IsOpen U ∧ U ∉ minimalBasis t } where
  toFun B := { U | U.1 ∈ B.1.1 }
  invFun S :=
    let Bset : Set (Set α) := minimalBasis t ∪ (Subtype.val '' S)
    have h : IsBasis Bset ∧ generateFrom Bset = t :=
      (isBasis_and_generates_iff t Bset).2 ⟨subset_union_left, by
        intro U hU
        rcases hU with hM | hS
        · obtain ⟨x, rfl⟩ := hM
          exact isOpen_minimalOpen t x
        · obtain ⟨U', _, rfl⟩ := hS
          exact U'.2.1⟩
    ⟨⟨Bset, h.1⟩, h.2⟩
  left_inv B := by
    have hiff := (isBasis_and_generates_iff t B.1.1).1 ⟨B.1.2, B.2⟩
    refine Subtype.ext (Subtype.ext ?_)
    ext U
    constructor
    · rintro (hM | ⟨U', hU', rfl⟩)
      · exact hiff.1 hM
      · exact hU'
    · intro hU
      by_cases hM : U ∈ minimalBasis t
      · exact Or.inl hM
      · exact Or.inr ⟨⟨U, hiff.2 hU, hM⟩, hU, rfl⟩
  right_inv S := by
    ext U
    constructor
    · intro h
      rcases h with hM | ⟨U', hU', hEq⟩
      · exact (U.2.2 hM).elim
      · exact (Subtype.ext hEq : U' = U) ▶ hU'
    · intro hU
      exact Or.inr ⟨U, hU, rfl⟩
```

Summing over all disjoint fibers yields the exact identity:

$$\boxed{\#(S) = \sum_{\mathcal{T} \in \text{Top}(S)} 2^{|\mathcal{T}|-|\mathcal{M}_\mathcal{T}|}}$$

```
omit [DecidableEq α] in
theorem minimalBasis_subset_opens : minimalBasis t ⊆ opensOf t := by
  rintro _ ⟨x, rfl⟩
  exact isOpen_minimalOpen t x

omit [DecidableEq α] in
/-- One fiber: bases generating `t` are a power set of redundant opens. -/
theorem card_fiber :
    Fintype.card (Fiber t) =
      2 ^ (ncard (opensOf t) - ncard (minimalBasis t)) := by
  classical
  rw [Fintype.card_congr (fiberEquiv t), Fintype.card_set]
```

```
  congr 1
  rw [← Nat.card_eq_fintype_card]
  change Nat.card ⇑({U : Set α | t.IsOpen U ∧ U ∉ minimalBasis t}) = _
  rw [Nat.card_coe_set_eq,
    show {U : Set α | t.IsOpen U ∧ U ∉ minimalBasis t} = opensOf t \ minimalBasis t by
      ext U; simp [opensOf, mem_diff]]
  exact ncard_diff (minimalBasis_subset_opens t)

omit [DecidableEq α] in
/-- Partition of all valid bases into fibers. -/
theorem card_valid_bases_sum_fiber :
    Fintype.card (ValidBasis α) =
      ∑ τ : TopologicalSpace α, Fintype.card (Fiber τ) := by
  rw [← Fintype.card_congr
    (Equiv.sigmaFiberEquiv (fun B : ValidBasis α => generateFrom B.1))]
  rw [Fintype.card_sigma]

omit [DecidableEq α] in
/-- Main theorem: the number of valid bases on an `N`-element set
    equals the sum over all topologies of `2^(|T| - |M_T|)`. -/
theorem card_valid_bases :
    Fintype.card (ValidBasis α) =
      ∑ τ : TopologicalSpace α,
        2 ^ (ncard (opensOf τ) - ncard (minimalBasis τ)) := by
  rw [card_valid_bases_sum_fiber]
  exact Finset.sum_congr rfl fun τ _ => card_fiber τ
```

`#print axioms card_valid_bases` reports `{propext, Classical.choice, Quot.sound}`. Choice enters because `IsOpen` is not a decidable predicate, so the `Fintype` instances on topologies and `card_fiber` use it.

---

4 Evaluation for Small N

N	Topologies $\lvert\mathrm{Top}(S)\rvert$	Basis computations	Total bases $\#(N)$
0	1 (trivial: $\{\emptyset\}$)	$\lvert\mathcal{T}\rvert = 1$, $\lvert\mathcal{M}_{\mathcal{T}}\rvert = 0$ $\implies 2^{1-0}$	2
1	1 ($\{\emptyset, \{1\}\}$)	$\lvert\mathcal{T}\rvert = 2$, $\lvert\mathcal{M}_{\mathcal{T}}\rvert = 1$ $\implies 2^{2-1}$	2
2	4 (discrete, indiscrete, 2 Sierpiński)	Discrete $2^{4-2} = 4$; indiscrete $2^{2-1} = 2$; Sierpiński ($\times 2$): $2 \times 2^{3-2} = 4$	10
3	29	1 discrete ($2^4 = 16$), 6 of size 6 ($6 \times 2^2 = 24$), etc.	226

(Note: If $\emptyset$ is excluded from bases by convention, divide each result by 2.)

---

5 Combinatorial Complexity and Bounds

5.1 The Combinatorial Obstacle

The formula above cannot be collapsed into an elementary closed form in N. Counting the number of topologies on N labeled points, $|\operatorname{Top}(S)|$ (**OEIS A000798**), is isomorphic to counting finite preorders on N elements, a classic problem with no known closed-form generating function [1, 2].

5.2 Asymptotics and Dominance

While an elementary exact formula does not exist, asymptotic analysis reveals that the sum is doubly exponential and overwhelmingly dominated by a single fiber: the **discrete topology** $\mathcal{T}_{\text{disc}} = \mathcal{P}(S)$.

1. **Discrete Fiber:** For $\mathcal{T}_{\text{disc}}$, $|\mathcal{T}| = 2^N$ and $\mathcal{M}_{\mathcal{T}} = \{\{x\} \mid x \in S\}$, so $|\mathcal{M}_{\mathcal{T}}| = N$. This single topology contributes:
$$2^{2^N - N} \text{ valid bases.}$$

2. **Upper Bound on Non-Discrete Fibers:** Any proper subtopology $\mathcal{T} \subsetneq \mathcal{P}(S)$ has at most $|\mathcal{T}| \le \frac{3}{4}2^N = 3 \cdot 2^{N-2}$ open sets [4, 6]. Using the Kleitman–Rothschild bound on the number of preorders $|\operatorname{Top}(S)| \le 2^{N(N-1)}$ [3], the sum of all other fibers is bounded by:
$$\sum_{\mathcal{T} \neq \mathcal{T}_{\text{disc}}} 2^{|\mathcal{T}| - |\mathcal{M}_{\mathcal{T}}|} \le 2^{N(N-1)} \cdot 2^{\frac{3}{4}2^N - 1}$$

3. **Asymptotic Behavior:**
$$2^{2^N - N} \;\le\; \#(N) \;\le\; 2^{2^N - N} + 2^{N^2 + \frac{3}{4}2^N}$$
As $N \to \infty$:
$$\#(N) \sim 2^{2^N - N}$$
with relative error bounded by $O\left(2^{-\frac{1}{4}2^N + N^2}\right) \to 0$.

---

6 Acknowledgments

6.1 AI-assisted development

The human author retains sole responsibility for the mathematical content, the choice of formalization route, and every formal claim in this work. Following standard publisher practice (e.g., COPE guidance on authorship and AI tools [8]), **no large language model is listed as a co-author** — authorship implies an accountability that automated systems cannot bear.

We gratefully acknowledge assistance from the following tools. None of these models has an official Hugging Face model card (they are closed, vendor-hosted systems); the citations below are the publisher model cards and the Cursor integration pages.

- **Cursor Grok 4.6 High Fast** [9] — primary agent for this note: Lean 4 / Mathlib formalization of the fiber identity (`CARDB.lean`), `lake build` repair, and drafting `CARDB.md`. Used in the Cursor agent environment at the **High** reasoning tier and **Fast** speed variant. Jointly trained by SpaceXAI and Cursor. Generated Lean was provisional until it compiled under the pinned toolchain (`leanprover/lean4:v4.30.0`).

- **Anthropic Claude Sonnet 5** [10] — initial Lean sketch of the fiber identity (`IsBasis`, `ValidBasis`, `fiberEquiv`, and the sum over topologies) and an early point-set write-up of the Alexandrov minimal-neighborhood argument. The sketch was unchecked; subsequent agents repaired it to a sorry-free Mathlib formalization under the pinned toolchain. Used in the Cursor agent environment.
- **Google Gemini 3.7 Flash** [11] — exploratory and parallel passes on the same combinatorial / point-set material (basis axioms, Alexandrov minimal neighborhoods, and the finite-enumeration identity).

All definitions, constructivity audits, and final prose were reviewed by the human author, who takes full responsibility for them.

6.2 Artifact availability

The development is at `github.com/catskillsresearch/scott_models/tree/main/CARDB`. Run `cd CARDB && lake build` for the sorry-free formalization. Run `cd CARDB && python3 build_pdf.py` to regenerate `CARDB.tex` and `CARDB.pdf` (title page, author, affiliation, and this GitHub path).

---

7 References

1. Alexandrov, P. (1937). Diskrete Räume. *Matematicheskii Sbornik*, 2(3), 501–519.
2. Benoumhani, M. (2006). The number of topologies on a finite set. *Journal of Integer Sequences*, 9(2), Article 06.2.6.
3. Erné, M., & Stege, K. (1991). Counting finite topologies and posets. *Order*, 8(3), 247–265.
4. Froemke, J., & Quackenbush, R. (1975). The spectrum of an equational class of lattices. *Pacific Journal of Mathematics*, 60(1), 49–54.
5. Kleitman, D. J., & Rothschild, B. L. (1975). Asymptotic enumeration of partial orders on a finite set. *Transactions of the American Mathematical Society*, 205, 205–220.
6. Stephen, D. (1968). Topology on finite sets. *The American Mathematical Monthly*, 75(7), 739–741.
7. OEIS Foundation Inc. (2026). The On-Line Encyclopedia of Integer Sequences. Sequences A000798 (*Number of topologies on n labeled points*) and A001035 (*Number of labeled partial orders on n points*). https://oeis.org.
8. Committee on Publication Ethics (COPE). *Authorship and AI tools: COPE position statement.* 2024. `https://publicationethics.org/guidance/cope-position/authorship-and-ai-tools`
9. SpaceXAI and Anysphere, Inc. *Grok 4.6* (High reasoning, Fast variant). Official model card (12 August 2026), `https://media.x.ai/v1/website/card-7f81d41b.pdf`; developer documentation, `https://docs.x.ai/developers/models/grok-4.6`; Cursor model page, `https://cursor.com/docs/models/grok-4-6`; Cursor announcement, `https://cursor.com/blog/grok-4-6` (accessed 2026). No official Hugging Face model card is published for this closed model.

10. Anthropic. *Claude Sonnet 5*. Official system card (30 June 2026), https://www.anthropic.com/claude-sonnet-5-system-card; system-card PDF, https://www-cdn.anthropic.com/480e0bb54327b9622282e9c39a83a4f490ed377e/Claude%20Sonnet%205%20System%20Card.pdf; product page, https://www.anthropic.com/claude/sonnet; announcement, https://www.anthropic.com/news/claude-sonnet-5; Cursor model page, https://cursor.com/docs/models/claude-sonnet-5 (accessed 2026). No official Hugging Face model card is published for this closed model.

11. Google DeepMind. *Gemini 3.7 Flash*. Official model card (13 August 2026), https://deepmind.google/models/model-cards/gemini-3-7-flash/; product announcement, https://blog.google/innovation-and-ai/models-and-research/gemini-models/introducing-gemi Gemini API model catalog, https://ai.google.dev/gemini-api/docs/models (accessed 2026). No official Hugging Face model card is published for this closed model.

A Complete Lean source

`CARDB.lean` in full (234 lines), checked by `lake build` against Lean 4 and Mathlib. The fiber lemma and the sum identity of Section 3 are proved here; `#print axioms card_valid_bases` reports {`propext`, `Classical.choice`, `Quot.sound`}.

```
/-
Copyright (c) 2026  Lars Warren Ericson.  All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github:  https://github.com/catskillsresearch/scott_models/CARDB/CARDB.lean
-/

/-
  Cardinality of topological bases on a finite set (CARDB).

  Claude sketch from CARDB.md, now a real Lake module. Definitions follow
  the paper; Mathlib's `IsTopologicalBasis` is the same two axioms plus
  `eq_generateFrom` relative to a fixed topology.
-/

import Mathlib.Topology.AlexandrovDiscrete
import Mathlib.Data.Fintype.BigOperators
import Mathlib.Data.Fintype.Powerset
import Mathlib.Data.Set.Card
import Mathlib.Data.Set.Finite.Basic
import Mathlib.Logic.Equiv.Sum

open Set TopologicalSpace
open scoped BigOperators

variable {α : Type*} [Fintype α] [DecidableEq α]

/-- A collection of subsets `B` is a topological basis if it covers `α`
    and satisfies the local intersection property. -/
def IsBasis (B : Set (Set α)) : Prop :=
  (⋃₀ B = univ) ∧
  ∀ {{U V}}, U ∈ B → V ∈ B → ∀ x ∈ U ∩ V, ∃ W ∈ B, x ∈ W ∧ W ⊆ U ∩ V

def ValidBasis (α : Type*) [Fintype α] := { B : Set (Set α) // IsBasis B }

/-- Open sets of `t`, as a family of subsets. This map is injective, so there
    are only finitely many topologies on a finite type. -/
def opensOf (t : TopologicalSpace α) : Set (Set α) := {U | t.IsOpen U}

omit [Fintype α] [DecidableEq α] in
```

```lean
theorem opensOf_injective :
    Function.Injective (opensOf : TopologicalSpace α → Set (Set α)) :=
  leftInverse_generateFrom.injective

noncomputable instance : DecidableEq (TopologicalSpace α) :=
  opensOf_injective.decidableEq

noncomputable instance : Fintype (TopologicalSpace α) :=
  Fintype.ofInjective opensOf opensOf_injective

noncomputable instance : DecidablePred (IsBasis : Set (Set α) → Prop) :=
  fun _ => Classical.dec _

noncomputable instance : Fintype (ValidBasis α) :=
  Subtype.fintype _

variable (t : TopologicalSpace α)

/-- Valid bases that generate exactly `t`: the fiber of `generateFrom` over `t`. -/
abbrev Fiber (t : TopologicalSpace α) :=
  { B : ValidBasis α // generateFrom B.1 = t }

/-- In a finite topological space, the minimal open neighborhood of `x`
    (Mathlib's `nhdsKer {x}` at topology `t`). -/
abbrev minimalOpen (x : α) : Set α :=
  @nhdsKer α t {x}

/-- The minimal basis `M_T` is the collection of all minimal open neighborhoods. -/
def minimalBasis : Set (Set α) :=
  range (minimalOpen t)

omit [Fintype α] [DecidableEq α] in
theorem mem_minimalOpen_self (x : α) : x ∈ minimalOpen t x :=
  @subset_nhdsKer α t {x} x (mem_singleton x)

omit [Fintype α] [DecidableEq α] in
theorem minimalOpen_subset_of_isOpen {x : α} {U : Set α}
    (hU : t.IsOpen U) (hx : x ∈ U) : minimalOpen t x ⊆ U :=
  @nhdsKer_minimal α t {x} U (singleton_subset_iff.mpr hx) hU

omit [DecidableEq α] in
/-- Finite spaces are Alexandrov-discrete, so `nhdsKer {x}` is open. -/
theorem isOpen_minimalOpen (x : α) : t.IsOpen (minimalOpen t x) :=
  letI := t
  isOpen_nhdsKer

omit [DecidableEq α] in
/-- The minimal basis alone generates the topology `t`. -/
theorem generateFrom_minimalBasis : generateFrom (minimalBasis t) = t := by
  ext U
  constructor
  · intro h
    letI := t
    induction h with
    | basic s hs =>
      obtain ⟨x, rfl⟩ := hs
      exact isOpen_minimalOpen t x
    | univ => exact isOpen_univ
    | inter _ _ _ _ hs ht => exact hs.inter ht
    | sUnion S _ hS => exact isOpen_sUnion fun s hs => hS s hs
  · intro hU
    have : U = ⋃₀ (minimalOpen t '' U) := by
      ext y
```

```
      constructor
      · intro hy
        exact ⟨minimalOpen t y, ⟨y, hy, rfl⟩, mem_minimalOpen_self t y⟩
      · rintro ⟨_, ⟨x, hx, rfl⟩, hy⟩
        exact minimalOpen_subset_of_isOpen t hU hx hy
    rw [this]
    exact GenerateOpen.sUnion _ fun V hV =>
      let ⟨x, _, hx⟩ := hV
      hx ▸ GenerateOpen.basic _ (mem_range_self x)

omit [Fintype α] [DecidableEq α] in
/-- `IsBasis` plus `generateFrom B = t` is Mathlib's `IsTopologicalBasis`. -/
theorem isTopologicalBasis_generateFrom {B : Set (Set α)} (hB : IsBasis B) :
    @IsTopologicalBasis α (generateFrom B) B := by
  letI := generateFrom B
  refine ⟨?_, hB.1, rfl⟩
  intro t₁ ht₁ t₂ ht₂ x hx
  exact hB.2 ht₁ ht₂ x hx

omit [DecidableEq α] in
/-- Core lemma: `B` generates `t` iff `B` contains the minimal basis
    and only contains open sets of `t`. -/
theorem isBasis_and_generates_iff (B : Set (Set α)) :
    (IsBasis B ∧ generateFrom B = t) ↔
    (minimalBasis t ⊆ B ∧ B ⊆ {U | t.IsOpen U}) := by
  constructor
  · rintro ⟨hB, rfl⟩
    constructor
    · rintro _ ⟨x, rfl⟩
      letI := generateFrom B
      have hb : IsTopologicalBasis B := isTopologicalBasis_generateFrom hB
      have hx : x ∈ minimalOpen (generateFrom B) x := mem_minimalOpen_self _ x
      have hUo : IsOpen (minimalOpen (generateFrom B) x) := isOpen_minimalOpen _ x
      obtain ⟨W, hW, hxW, hWU⟩ := hb.exists_subset_of_mem_open hx hUo
      have hUW : minimalOpen (generateFrom B) x ⊆ W :=
        minimalOpen_subset_of_isOpen _ (isOpen_generateFrom_of_mem hW) hxW
      rwa [← Subset.antisymm hWU hUW]
    · intro U hU
      exact isOpen_generateFrom_of_mem hU
  · rintro ⟨h_min, h_sub⟩
    constructor
    · constructor
      · ext x
        simp only [mem_sUnion, mem_univ, iff_true]
        exact ⟨minimalOpen t x, h_min ⟨x, rfl⟩, mem_minimalOpen_self t x⟩
      · intro U V hU hV x hx
        refine ⟨minimalOpen t x, h_min ⟨x, rfl⟩, mem_minimalOpen_self t x, ?_⟩
        intro y hy
        exact ⟨minimalOpen_subset_of_isOpen t (h_sub hU) hx.1 hy,
          minimalOpen_subset_of_isOpen t (h_sub hV) hx.2 hy⟩
    · refine le_antisymm ?_ ?_
      · exact generateFrom_anti h_min |>.trans (generateFrom_minimalBasis t).le
      · exact le_generateFrom_iff_subset_isOpen.2 h_sub

/-- The fiber over a topology `t` is equivalent to the power set
    of the redundant open sets. -/
def fiberEquiv (t : TopologicalSpace α) :
    Fiber t ≃
      Set { U : Set α // t.IsOpen U ∧ U ∉ minimalBasis t } where
  toFun B := { U | U.1 ∈ B.1.1 }
  invFun S :=
    let Bset : Set (Set α) := minimalBasis t ∪ (Subtype.val '' S)
    have h : IsBasis Bset ∧ generateFrom Bset = t :=
```

```
      (isBasis_and_generates_iff t Bset).2 ⟨subset_union_left, by
        intro U hU
        rcases hU with hM | hS
        · obtain ⟨x, rfl⟩ := hM
          exact isOpen_minimalOpen t x
        · obtain ⟨U', _, rfl⟩ := hS
          exact U'.2.1⟩
    ⟨⟨Bset, h.1⟩, h.2⟩
  left_inv B := by
    have hiff := (isBasis_and_generates_iff t B.1.1).1 ⟨B.1.2, B.2⟩
    refine Subtype.ext (Subtype.ext ?_)
    ext U
    constructor
    · rintro (hM | ⟨U', hU', rfl⟩)
      · exact hiff.1 hM
      · exact hU'
    · intro hU
      by_cases hM : U ∈ minimalBasis t
      · exact Or.inl hM
      · exact Or.inr ⟨⟨U, hiff.2 hU, hM⟩, hU, rfl⟩
  right_inv S := by
    ext U
    constructor
    · intro h
      rcases h with hM | ⟨U', hU', hEq⟩
      · exact (U.2.2 hM).elim
      · exact (Subtype.ext hEq : U' = U) ▸ hU'
    · intro hU
      exact Or.inr ⟨U, hU, rfl⟩

omit [DecidableEq α] in
theorem minimalBasis_subset_opens : minimalBasis t ⊆ opensOf t := by
  rintro _ ⟨x, rfl⟩
  exact isOpen_minimalOpen t x

omit [DecidableEq α] in
/-- One fiber: bases generating `t` are a power set of redundant opens. -/
theorem card_fiber :
    Fintype.card (Fiber t) =
      2 ^ (ncard (opensOf t) - ncard (minimalBasis t)) := by
  classical
  rw [Fintype.card_congr (fiberEquiv t), Fintype.card_set]
  congr 1
  rw [← Nat.card_eq_fintype_card]
  change Nat.card ↥({U : Set α | t.IsOpen U ∧ U ∉ minimalBasis t}) = _
  rw [Nat.card_coe_set_eq,
    show {U : Set α | t.IsOpen U ∧ U ∉ minimalBasis t} = opensOf t \ minimalBasis t by
      ext U; simp [opensOf, mem_diff]]
  exact ncard_diff (minimalBasis_subset_opens t)

omit [DecidableEq α] in
/-- Partition of all valid bases into fibers. -/
theorem card_valid_bases_sum_fiber :
    Fintype.card (ValidBasis α) =
      ∑ τ : TopologicalSpace α, Fintype.card (Fiber τ) := by
  rw [← Fintype.card_congr
    (Equiv.sigmaFiberEquiv (fun B : ValidBasis α => generateFrom B.1))]
  rw [Fintype.card_sigma]

omit [DecidableEq α] in
/-- Main theorem: the number of valid bases on an `N`-element set
    equals the sum over all topologies of `2^(|T| - |M_T|)`. -/
theorem card_valid_bases :
```

```
  Fintype.card (ValidBasis α) =
    ∑ τ : TopologicalSpace α,
      2 ^ (ncard (opensOf τ) - ncard (minimalBasis τ)) := by
rw [card_valid_bases_sum_fiber]
exact Finset.sum_congr rfl fun τ _ => card_fiber τ
```